

A Content-Based Recommendation System for Precedent Retrieval in Hindu Marriage Act Divorce Cases

Aryan, [Team Member 2], [Team Member 3], [Team Member 4], [Team Member 5], [Team Member 6]

*Department of Computer Science and Engineering
VIT Bhopal University, Bhopal, Madhya Pradesh, India*

Abstract

People involved in matrimonial disputes under the Hindu Marriage Act, 1955, usually have no quick way of checking how similar cases were decided in the past. This paper describes a content-based recommendation tool built to close that gap: a user types in a short description of a case's facts, and the system returns past divorce judgments that resemble it most closely, along with their outcomes. The pipeline has two parts. Offline, a set of Indian court judgments is collected, filtered down to family-law divorce matters, tagged by ground of divorce, and converted into a TF-IDF representation of the cleaned case facts. Online, a user's query is cleaned the same way, compared against the stored case vectors using cosine similarity, and optionally narrowed by a chosen ground of divorce before the top matches are shown, with verdicts and shared keywords, through a Streamlit interface. The method is a fairly standard content-based filtering setup, the kind used for recommending similar products or articles, applied here to case facts instead. This paper covers how the dataset was built and cleaned, how the recommendation engine works, how the interface is laid out, and where the current version falls short, before pointing to some ways it could be extended.

Keywords: *Recommendation systems, legal case retrieval, TF-IDF, cosine similarity, content-based filtering, Hindu Marriage Act, natural language processing.*

1. Introduction

Matrimonial cases under the Hindu Marriage Act, 1955 can be filed on several grounds, cruelty, desertion, adultery, irretrievable breakdown of marriage, mutual consent, and a few others, and courts lean heavily on precedent while deciding them. A litigant, a paralegal, or a law or CS student trying to find factually similar older cases usually has to search a case database by hand, and that only works well if you already know the right search terms. That is the gap this project tries to fill: given a plain description of a case's facts, the system should be able to pull up the previously decided cases that look most similar, and show how they were resolved.

The idea borrows from ordinary recommendation problems, recommending similar products or articles, where a document gets turned into a feature vector and similarity between vectors decides what gets suggested. Here, a judgment's facts become the item profile, a user's case description becomes the query, and cases are ranked by how close they sit to that query.

The rest of the paper is laid out as follows. Section 2 covers related work. Section 3 walks through the dataset, the preprocessing, and the recommendation approach, including the system architecture. Section 4 presents the results and the working interface. Section 5 discusses what the results actually show and where the system struggles. Section 6 concludes, and Section 7 lists what could be done next.

2. Literature Review

Legal recommender systems go back further than one might expect. Winkels et al. [1] built one of the earlier examples for a Dutch legislative portal, where the system pointed users toward related statutes by

turning the references already present in a case into machine-readable links. Their approach worked well for cases that already cited relevant law, but it relied on citation structure rather than on what the case was actually about, so two factually similar cases with no shared citations would not be linked at all.

A more practitioner-oriented system is Quick Check, described by Thomas et al. [3]. It reads a litigation brief, extracts the legal arguments from it, and recommends case law opinions using a mix of full-text search, citation-network analysis, and clickstream data, with a ranking model trained on over ten thousand human annotations. It is a strong system, but it needs a large annotated corpus and an established citation graph behind it, resources that are simply not available for a small, manually built dataset of the kind used in this project.

Closer to the approach taken here, Dhanani, Mehta and Rana [2] built a legal document recommender for Indian court judgments using a dictionary-based representation instead of the heavier Doc2Vec embeddings used in earlier systems, arguing that the sheer size and noise of a legal vocabulary makes embedding models slow to train without necessarily improving the results. Makawana and Mehta [9] took the representation question further and proposed a paragraph-level filtering technique that treats a judgment as a network of paragraphs rather than one flat block of text, using that structure to decide which parts of two documents are worth comparing at all.

Other recent work moves past plain word-overlap methods altogether. Dhani et al. [4] built a knowledge graph out of Indian court judgments and framed case similarity as a graph problem, linking cases through shared statutes, sections and cited precedents rather than shared vocabulary. Oliveira and Sperandio Nascimento [6] fine-tuned several transformer models, BERT, GPT-2, RoBERTa and LLaMA, on over two hundred thousand Brazilian labour-court cases and clustered the resulting embeddings; the domain-tuned LLaMA model produced noticeably tighter, more accurate clusters than any of the classic NLP baselines they compared it against. Ostendorff et al. [7] ran a broader comparison still, testing twenty-seven different document representation methods, text-based, citation-based and hybrid, on a benchmark of US case law they built themselves, and found that no single representation dominates across the board, which is part of why the present project sticks with a simple, explainable TF-IDF baseline rather than reaching straight for a transformer model. Li et al. [8] approached the retrieval side of the problem differently again, releasing LeCaRDv2, a large Chinese legal case retrieval benchmark with over fifty thousand candidate cases judged on characterization, penalty and procedure rather than plain topical overlap, which is a useful reminder that matching subject matter alone does not capture what makes two cases relevant to each other.

Large language models are the newest addition to this line of work. Shu et al. [5] introduced LawLLM, a model built specifically for the US legal system that handles similar-case retrieval, precedent recommendation and judgment prediction as three separate tasks, on the grounds that researchers often blur the line between a similar case and a precedent case when really the two call for different techniques. Deroy et al. [10] look at the problem from a data-organisation angle instead, discussing how legal ontologies, citation graphs and TF-IDF based topic extraction can be combined to make an otherwise unstructured mass of judgments usable by both practitioners and downstream systems.

Taken together, this body of work shows a fairly clear split: citation-graph and knowledge-graph methods work well once a case corpus has rich cross-references, while transformer and LLM-based methods need either a large labelled dataset or heavy compute, or both. Neither condition holds for a small, freshly curated set of Hindu Marriage Act judgments with no existing citation graph. A TF-IDF and cosine-similarity baseline, of the kind used by Dhanani et al. [2] in the Indian legal setting, is therefore a reasonable starting point: it needs no training data beyond the corpus itself, it stays fully explainable, and

it can be built and tested quickly enough to fit within a course project's timeline. That is the gap this paper's system sits in, and Section 3 describes how it was built.

3. Methodology

The system is built as a content-based recommendation pipeline with a clear split between an offline stage, run once to prepare the case corpus and its feature representation, and an online stage that answers queries as they come in. The subsections below walk through each part of that pipeline.

3.1 Dataset Collection and Filtering

The base corpus came from a large open Indian legal-judgment dataset covering cases from 2016 to 2020 and 2021 to 2025, with fields such as case title, case type, facts summary, issues raised, acts summary, full judgment text and keywords. To pull out matrimonial and divorce matters, records were filtered on two signals at once: a `case_type` tag equal to “Family Law”, and a keyword search across the facts, issues, acts-summary, full-text and keyword fields for terms like “Hindu Marriage Act”, “divorce”, “irretrievable breakdown”, “judicial separation”, “mutual consent”, “cruelty” and “desertion”. Cases matching both filters, Family Law and a divorce-related keyword, were treated as the most reliable starting subset.

That subset was then checked case by case. Titles that clearly were not divorce decrees, service, criminal or general civil matters that just happened to mention a word like “cruelty” in passing, were dropped. A separate “borderline” group, matrimonial-adjacent cases such as custody or habeas corpus matters connected to a marriage but not themselves divorce decrees, was set aside for later review instead of being folded into the main corpus. What is left after this cleaning pass is stored as `clean_divorce_cases.csv`.

3.2 Preprocessing and Ground-of-Divorce Tagging

Every case that made it through filtering was tagged with a `ground_of_divorce` label using keyword rules over its facts and issues text. This gives each case a structured field the interface can filter on later, on top of free-text similarity search. The tagged corpus is saved as `tagged_divorce_cases.csv`.

For the text-similarity part, case facts went through a standard NLP cleanup: lower-casing, stripping non-alphabetic characters, tokenizing, removing English stopwords using NLTK's list, and lemmatizing with NLTK's WordNetLemmatizer. The output is a `cleaned_facts` field for each case, and that field is what feeds the vector representation.

3.3 Feature Representation, TF-IDF

The cleaned facts of every case are turned into a TF-IDF matrix using scikit-learn's `TfidfVectorizer`, fitted once over the whole corpus when the app starts and cached so it only runs once per session. TF-IDF gives more weight to terms that show up often in one case but rarely across the corpus as a whole, which pushes the distinctive vocabulary of a case's facts, its specific behaviours and grounds, ahead of the generic legal boilerplate that shows up in almost every judgment.

3.4 Similarity Computation and Ranking

When a query comes in, the user's free-text description is cleaned with the same pipeline and turned into a vector using the already-fitted TF-IDF vectorizer. Cosine similarity is then computed between that query vector and every case vector in the corpus, or in the filtered subset if a ground of divorce was selected, using scikit-learn's `cosine_similarity`. Cases are sorted by similarity score and the top N are returned, with N chosen by the user in a range of one to ten. For each recommended case, the system also works out

which terms it shares most heavily with the query, by taking an element-wise product of the two TF-IDF vectors, and shows those as the “matched terms” so the user can see why a case turned up.

3.5 System Architecture and Interface

The whole thing is written in Python and served through a Streamlit web app with two tabs, a Recommender tab and an About tab. In the Recommender tab, a user can pick a ground of divorce from a sidebar filter, type a description of case facts, and choose how many results to see. Results show up as cards with the case title, ground of divorce, a colour-coded verdict badge, similarity score, and matched keywords, plus a bar chart comparing similarity across the returned cases and an expandable section with the full judgment text. The About tab documents the pipeline itself, data collection, ground tagging, preprocessing, TF-IDF vectorization, cosine-similarity search, the interface, and shows a few summary numbers for the corpus: total case count, number of distinct grounds of divorce, and number of distinct verdict types.

On the software side, the stack is Python, pandas, NLTK, scikit-learn and Streamlit, with the corpus stored as CSV files. Development happened on a machine with an Intel Core i5-13420H, 16 GB RAM and an NVIDIA RTX 4050, though the current pipeline runs entirely on CPU and does not use the GPU at all.

4. Results

[Space reserved for Results.]

Suggested content to fill in once evaluation is complete:

- *Final corpus size and the number of distinct grounds of divorce and verdict types (readable directly off the About tab metrics).*
- *Screenshots of the Streamlit interface: query input, filtered results, similarity bar chart, matched-keyword chips.*
- *An example query with its top-N recommended cases, similarity scores and verdicts.*
- *Any evaluation done, even a small spot-check of whether the top matches look factually sensible, or a precision-at-k number on a hand-labeled sample.*

5. Discussion

TF-IDF with cosine similarity is a lightweight approach, and its biggest advantage is that it is easy to explain. Every recommendation can be traced back to the exact overlapping words that produced it, which matters in a legal setting where a user needs to check a suggestion rather than just trust it. Pairing that with a structured ground-of-divorce filter also lets someone narrow the search using a legally meaningful attribute instead of relying on text overlap alone, something none of the pure keyword or citation-graph systems reviewed in Section 2 do quite the same way.

That said, a bag-of-words representation has real limits. It cannot tell that “cruelty” and “mental harassment” often mean roughly the same thing in this context, and it struggles when two cases are legally equivalent but phrased very differently. Reasoning about how facts relate to each other is the kind of thing a trained legal-NLP model or an LLM, like the ones reviewed in Section 2, is better suited for. The manual, keyword-assisted filtering used while building the corpus is also a likely source of selection bias, and the

corpus itself is still small next to the full space of reported Hindu Marriage Act judgments, so it is hard to say with much confidence how well the results would generalize beyond it.

6. Conclusion

This paper described a content-based recommendation system for finding precedent Hindu Marriage Act divorce cases similar to a user's description of their own case. It combines a curated, ground-of-divorce-tagged corpus with a TF-IDF and cosine-similarity engine, wrapped in a Streamlit interface that returns ranked, explainable recommendations along with case outcomes. Even without a heavier deep-learning model, this kind of simple, transparent pipeline already gives a useful starting point for finding precedent in a fairly narrow legal domain, and it leaves clear room for the richer NLP techniques discussed as future work.

7. Future Scope

- Grow the corpus past the current manually-verified subset to cover more, and more varied, Hindu Marriage Act judgments.
- Swap or extend the TF-IDF representation for contextual embeddings, sentence-transformer or legal-domain language models, to capture meaning rather than just word overlap.
- Bring in structured legal metadata, court level, year, cited sections and precedents, as extra ranking signals alongside text similarity.
- Add a real evaluation step, precision at k on a small hand-labeled set of query-case pairs, to measure recommendation quality instead of assuming it.
- Move the ground-of-divorce tagging from keyword rules to a trained classifier, and extend the same pipeline to other parts of family law.

References

- [1] R. Winkels , A. Boer, B. Vredereg , and A. van Someren , "Towards a Legal Recommender System ," JURIX 2014 — https://www.researchgate.net/publication/269702976_Towards_a_Legal_Recommender_System
- [2] J. Dhanani , R. Mehta , and D. P. Rana , "Legal document recommendation system: a dictionary based approach ," IJWIS, 2021 — <https://www.emerald.com/insight/content/doi/10.1108/ijwis-02-2021-0015/full/html>
- [3] M. Thomas et al. , "Quick Check: A Legal Research Recommendation System ," NLLP@KDD 2020 — https://www.academia.edu/112515830/Quick_Check_A_Legal_Research_Recommendation_System
- [4] J.S. Dhani , R. Bhatt , B. Ganesan , P. Sirohi , and V. Bhatnagar , "Similar Cases Recommendation using Legal Knowledge Graphs ," arXiv: 2107.04771 — <https://arxiv.org/abs/2107.04771> (HTML: <https://arxiv.labs.arxiv.org/html/2107.04771>)
- [5] D. Shu , H. Zhao , X. Liu , D. Demeter , M. Du , and Y. Zhang , "LawLLM: Law Large Language Model for the US Legal System ," arXiv: 2407.21065 — <https://arxiv.org/pdf/2407.21065>
- [6] R. Souza de Oliveira and E. G. Sperandio Nascimento , "Analysing similarities between legal court documents using natural language processing approaches based on transformers ," PLOS ONE , 2025 — <https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0320244>
- [7] M. Ostendorff, E. Ash, T. Ruas, B. Gipp , J. Moreno-Schneider , and G. Rehm , "Evaluating Document Representations for Content-based Legal Literature Recommendations ," ICAIL 2021 — <https://arxiv.org/pdf/2104.13841>

- [8] H. Li , Y. Shao , Y. Wu , Q. Ai , Y. Ma , and Y. Liu , "LeCaRDv2: A Large-Scale Chinese Legal Case Retrieval Dataset ," arXiv:2310.17609 — <https://arxiv.org/pdf/2310.17609>
- [9] M. Makawana and R. G. Mehta , "A novel network-based paragraph filtering technique for legal document similarity analysis ," Artificial Intelligence and Law , 2023 — <https://link.springer.com/article/10.1007/s10506-023-09375-6>
- [10] A. Deroy , N. K. Bailung , K. Ghosh , S. Ghosh , and A. Chakraborty , "Artificial Intelligence (AI) in Legal Data Mining ," arXiv:2405.14707 — <https://arxiv.org/pdf/2405.14707>
- [11] F. Pedregosa et al. , "Scikit-learn: Machine Learning in Python ," JMLR, 2011 — <https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>
- [12] S. Bird, E. Klein , and E. Loper , *Natural Language Processing with Python (NLTK)* , O'Reilly, 2009 — <https://www.nltk.org/book/>
- [13] Streamlit Inc. , "Streamlit Documentation" — <https://docs.streamlit.io>
- [14] K. Kawshik , "Information-Retrieval-from-Legal-data" <https://github.com/kawshik8/Information-Retrieval-from-Legal-data>
- [15] OpenNyAI , "Opennyai" — <https://github.com/OpenNyAI/Opennyai>
- [16] Exploration-Lab , "CJPE: Court Judgment Prediction and Explanation" — <https://github.com/Exploration-Lab/CJPE>
- [17] J. George , "Application of NLP in Indian Judiciary" — <https://github.com/justine-george/Application-of-NLP-in-Indian-Judiciary>
- [18] A. Gabhi , "Predicting Court Judgement Decisions using NLP" — <https://github.com/agabhi017/Predicting-Court-Judgement-Decisions-using-NLP>
- [19] Scheggetta , "coliee24" — <https://github.com/Scheggetta/coliee24>